Polytech Dijon

5A Informatique et Electronique

TP RT-Therm

Harmony V3 AND FreeRTOS

Done by :

MAMBO MOTSOU JUNIOR D.
FONKOUA CHARLE LUCE SORIANE

Supervised by :

Mr. B. Heyrman

Year 2025–2026

Sommaire

1 Introduction **4**

2 Présentation des ressources du projet **5**

2.1 Microcontrôleur et carte de développement 5

2.2 Framework Harmony V3 6

2.3 FreeRTOS 7

2.4 FreeRTOS Queues 8

3 Architecture de RT-Therm V1 **10**

3.1 Principe général 10

3.2 Description des tâches 10

3.2.1 Tâche d'acquisition de la température 11

3.2.2 Tâche de conversion de la température . . . 12

3.2.3 Tâche d'affichage 13

3.2.4 Communication et synchronisation 14

4 Architecture de RT-Therm V2 **16**

4.1 Problématique et motivation 16

4.2 Principe général de RT-Therm V2 16

4.2.1 Rôle central du sémaphore binaire 17

4.2.2 Coordination entre les tâches 18

4.2.3 Gestion du temps et horodatage des mesures 19

4.2.4 Structure des données échangées 19

4.2.5 Interaction utilisateur et changement d'unité 20

4.2.6 Avantages de la solution retenue 21

4.3 Comparaison entre RT-Therm V1 et RT-Therm V2 22

5 Conclusion Générale **23**

Table des Figures

2.1 Carte Explorer 16/32 5
2.2 Framework Harmony V3 6
2.3 Logo de FreeRTOS 7
2.4 Principe de fonctionnement d'une queue FreeRTOS 8

3.1 Vue d'ensemble 10
3.2 AQUISITION 11
3.3 Tâche de conversion 12
3.4 Tâche de display 13
3.5 Display Temperature 14
3.6 Schema fonctionnel des tâches 15

4.1 Acquisition Config 17
4.2 Libération du sémaphore par la tâche d'acquisition après une conversion ADC valide 18
4.3 Attente et prise du sémaphore par la tâche d'analyse avant le traitement 18
4.4 Structure des données échangées 20
4.5 Structure des données échangées 21

Chapter 1

Introduction

L'objectif de ce travail pratique est de concevoir une application temps réel basée sur FreeRTOS, à partir d'un projet de thermomètre initialement développé en programmation séquentielle. Cette transition vers une architecture temps réel vise à améliorer la structuration logicielle, la réactivité du système et la gestion concurrente des différentes fonctionnalités.

L'application est implémentée sur un microcontrôleur PIC32MZ et s'appuie sur le framework logiciel Harmony V3 fourni par Microchip. Deux architectures distinctes ont été développées et comparées :

- **RT-Therm V1** : une architecture reposant sur trois tâches périodiques indépendantes, communiquant entre elles à l'aide de files d'attente (queues).
- **RT-Therm V2** : une architecture dans laquelle une seule tâche est explicitement temporisée, tandis que les autres sont synchronisées par des mécanismes de communication inter-tâches.

Ce compte-rendu présente les ressources matérielles et logicielles utilisées, l'architecture logicielle mise en place ainsi que les choix de conception retenus pour chacune des deux versions de l'application.

Chapter 2

Présentation des ressources du projet

Cette partie décrit les différentes ressources matérielles et logicielles intégrées au projet, ainsi que leur rôle dans le fonctionnement global du thermomètre temps réel.

2.1 Microcontrôleur et carte de développement

Le projet est basé sur le microcontrôleur **PIC32MZ2048EFH100**, monté sur la carte de développement **Explorer 16/32**. Ce microcontrôleur 32 bits dispose de ressources importantes, notamment une fréquence de fonctionnement élevée, une grande capacité mémoire et de nombreux périphériques intégrés, ce qui le rend particulièrement adapté à l'exécution d'un système d'exploitation temps réel tel que FreeRTOS.

La carte Explorer 16/32 facilite l'accès aux périphériques externes nécessaires au projet, notamment :

- le capteur de température,
- l'écran LCD,
- les différentes interfaces de communication utilisées par l'application.

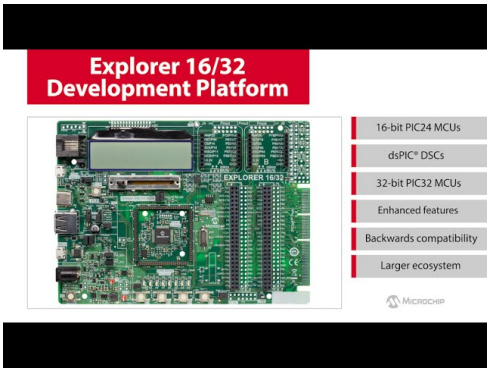


Figure 2.1: Carte Explorer 16/32

2.2 Framework Harmony V3

Harmony V3 est le framework logiciel officiel fourni par Microchip pour le développement d'applications embarquées sur les microcontrôleurs de la famille PIC32. Dans ce projet, il constitue un élément central de l'architecture logicielle, servant de socle à l'ensemble de l'application.

L'objectif principal de Harmony V3 est de simplifier et de structurer le développement logiciel, en particulier pour des projets complexes ou lors d'une première utilisation d'un microcontrôleur associé à un système temps réel.

Concrètement, Harmony V3 permet :

- **La configuration graphique des périphériques** à l'aide de MCC (MPLAB Code Configurator). L'utilisateur peut activer et paramétrer les modules matériels (ADC pour la mesure de température, PMP pour l'écran LCD, gestion des horloges, interruptions, etc.) sans intervenir directement sur les registres du microcontrôleur.
- **La génération automatique du code bas niveau** correspondant à la configuration sélectionnée. Ce code inclut l'initialisation matérielle, les drivers et les fonctions nécessaires à l'utilisation correcte des périphériques.
- **La gestion cohérente des drivers et services**, garantissant une compatibilité entre les différents modules du microcontrôleur (horloges, accès mémoire, interruptions et intégration du RTOS).

Dans le cadre de ce travail pratique, Harmony V3 présente un avantage majeur : il permet l'intégration native de FreeRTOS. L'ajout du composant *CORE* dans la configuration entraîne automatiquement la proposition d'intégrer un système d'exploitation temps réel, ce qui simplifie considérablement la création, la configuration et la gestion des tâches.



Figure 2.2: Framework Harmony V3

2.3 FreeRTOS

FreeRTOS est un système d'exploitation temps réel (*RTOS – Real-Time Operating System*) léger et largement répandu dans le domaine des systèmes embarqués. Dans le cadre d'une première utilisation, son objectif principal est de fournir une organisation claire et structurée du logiciel en plusieurs entités indépendantes, appelées **tâches**, capables de s'exécuter de manière concurrente.

Dans une application embarquée classique sans RTOS, le programme est généralement organisé autour d'une boucle principale unique, souvent appelée *super loop*. Cette approche montre rapidement ses limites lorsque plusieurs fonctionnalités doivent être exécutées périodiquement, à des fréquences différentes, ou lorsqu'un temps de réponse précis est requis. FreeRTOS répond à ces contraintes en introduisant une architecture temps réel **déterministe**, dans laquelle chaque fonctionnalité du système est isolée au sein d'une tâche dédiée.

Dans ce projet, FreeRTOS est utilisé afin de :

- structurer l'application en plusieurs tâches indépendantes correspondant aux différentes fonctions du thermomètre (acquisition de la température, traitement des données et affichage),
- assurer l'ordonnancement automatique des tâches par l'intermédiaire de l'ordonnanceur du RTOS,
- permettre l'utilisation de temporisations précises exprimées en millisecondes,
- garantir la synchronisation et l'échange sécurisé de données entre les tâches.

Chaque tâche FreeRTOS est associée à une fonction spécifique du système et dispose de son propre contexte d'exécution. L'ordonnanceur de FreeRTOS détermine, à chaque instant, quelle tâche doit être exécutée en fonction de son état (active, bloquée ou en attente) ainsi que de sa priorité. Ce mécanisme permet une utilisation efficace des ressources du processeur tout en assurant un comportement prévisible et reproductible de l'application.



Figure 2.3: Logo de FreeRTOS

2.4 FreeRTOS Queues

L'utilisation de *queues* (files d'attente) constitue un mécanisme fondamental et fiable pour le transfert de données entre tâches dans un environnement basé sur un système d'exploitation temps réel. Contrairement à une application *bare metal*, dans laquelle le programme s'exécute de manière séquentielle, un RTOS impose une gestion rigoureuse des données partagées afin d'éviter les conflits d'accès et de garantir un comportement déterministe.

Une queue FreeRTOS est une zone mémoire gérée par le noyau du RTOS et organisée selon le principe FIFO (*First-In, First-Out*). Elle permet de stocker directement des données (et non des pointeurs par défaut), dont la taille est définie lors de la création de la queue. Lorsqu'une tâche effectue une opération de réception (*receive*), l'élément le plus ancien de la file est extrait, libérant ainsi un emplacement mémoire. À l'inverse, lorsqu'une tâche envoie une donnée (*send*), celle-ci est ajoutée à la fin de la file, sous réserve que la queue ne soit pas pleine.

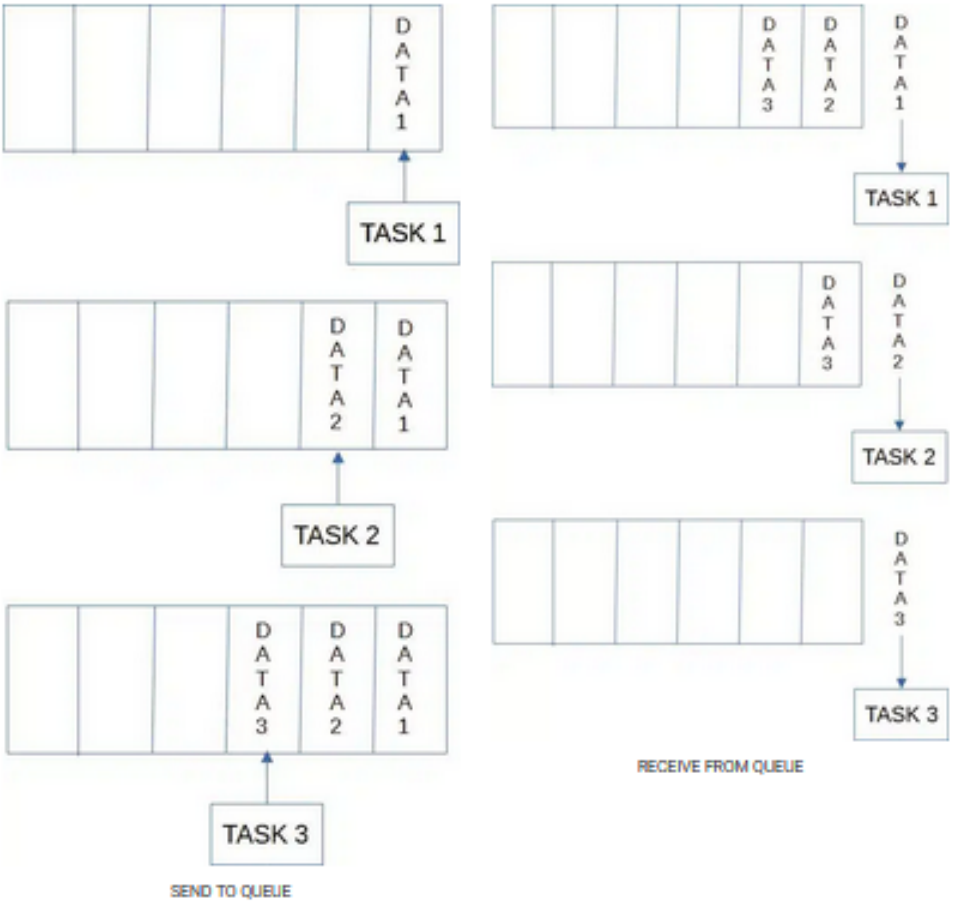


Figure 2.4: Principe de fonctionnement d'une queue FreeRTOS

L'ensemble des opérations liées aux queues est pris en charge par le noyau de FreeRTOS. Celui-ci assure automatiquement la synchronisation entre les tâches, l'intégrité des données échangées ainsi que le respect strict de l'ordre d'insertion des éléments. Ce mécanisme permet d'éviter l'utilisation explicite de sections critiques ou de sémaphores pour le simple transfert d'informations, tout en améliorant la lisibilité et la robustesse du code.

Dans le cadre de ce projet, les queues sont utilisées comme principal moyen de communication entre les différentes tâches de l'application :

- la tâche d'acquisition mesure la température et envoie la valeur brute dans une première queue ;
- la tâche de conversion récupère cette valeur, effectue le traitement nécessaire, puis transmet une chaîne de caractères formatée dans une seconde queue ;
- la tâche d'affichage lit cette chaîne de caractères et l'affiche sur l'écran LCD.

Afin de faciliter la compréhension du fonctionnement global de l'application, un schéma illustrant les interactions entre les tâches et les différentes queues est présenté en annexe. Ce schéma permet de visualiser clairement le flux de données et le rôle de chaque tâche dans l'architecture FreeRTOS mise en place.

Chapter 3

Architecture de RT-Therm V1

3.1 Principe général

La version RT-Therm V1 repose sur une architecture basée sur trois tâches distinctes, chacune dédiée à une fonction précise du thermomètre. Ce découpage fonctionnel permet de séparer clairement les responsabilités et d'exploiter pleinement les mécanismes offerts par FreeRTOS.

L'objectif principal de cette organisation est de transformer une application initialement séquentielle en une application temps réel modulaire, plus lisible et plus robuste.

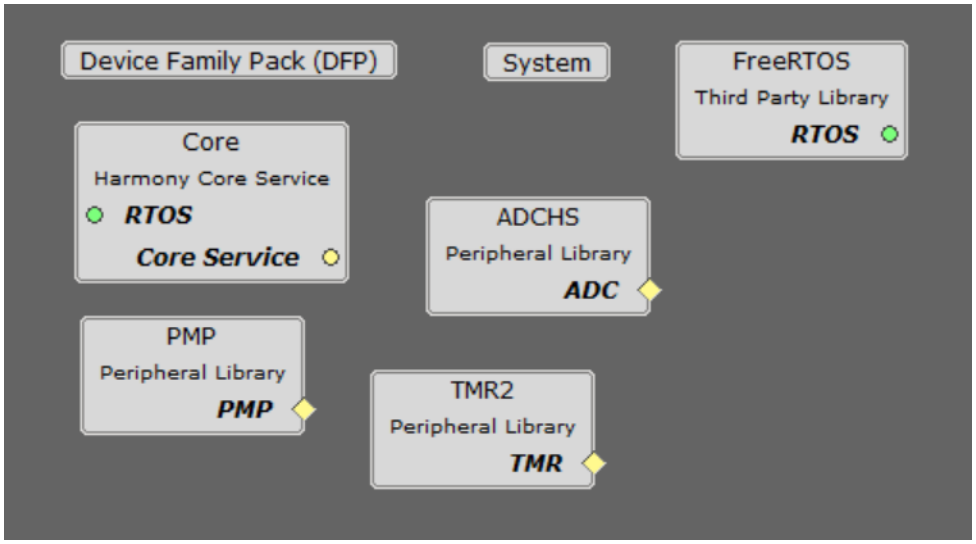


Figure 3.1: Vue d'ensemble

3.2 Description des tâches

Chaque tâche est dédiée à une fonction précise du thermomètre et s'exécute de manière concurrente sous le contrôle de l'ordonnanceur de FreeRTOS. Cette séparation fonctionnelle permet d'améliorer la lisibilité du code, la modularité de l'application et la gestion temporelle des différentes opérations.

Les trois tâches possèdent la même priorité et sont exécutées de façon périodique avec

une période de 500 ms.

3.2.1 Tâche d'acquisition de la température

La tâche d'acquisition de la température est responsable de la lecture périodique de la valeur brute fournie par le capteur de température via le convertisseur analogique-numérique (ADC). Cette tâche constitue le point d'entrée des données dans l'architecture temps réel du thermomètre.

Son implémentation repose sur le module ADC haute vitesse (*ADCHS*) du microcontrôleur PIC32MZ, configuré à l'aide du framework Harmony V3. Afin de garantir une architecture claire et modulaire, cette tâche est exclusivement dédiée à l'acquisition de la donnée, sans effectuer de traitement ou de conversion.

Lors de l'initialisation (`AQUISITION_Initialize`), une file d'attente FreeRTOS est créée à l'aide de la fonction `xQueueCreate`. Cette file d'attente permet de stocker les valeurs de température brutes (de type `uint16_t`) et d'assurer la communication avec la tâche de conversion. Le module ADCHS est également initialisé à cette étape.

La fonction `AQUISITION_Tasks` implémente une machine à états simple composée de deux états :

- **AQUISITION_STATE_INIT** : état d'initialisation de la tâche, permettant de préparer le passage vers l'état de service.
- **AQUISITION_STATE_SERVICE_TASKS** : état principal dans lequel l'acquisition de la température est effectuée.

```
case AQUISITION_STATE_SERVICE_TASKS:
{


    ADCHS_GlobalEdgeConversionStart();

    if(ADCHS_ChannelResultIsReady(ADCHS_CH4)){

        uint16_t Buffer = ADCHS_ChannelResultGet(ADCHS_CH4);

        if( myQueue != 0 ){

            xQueueSend(myQueue,&Buffer,10);
            GPIO_RA2_Toggle();

        }
```

Figure 3.2: AQUISITION

Dans l'état de service, une conversion ADC globale est déclenchée à l'aide de la fonction `ADCHS_GlobalEdgeConversionStart`. Une fois la conversion terminée, la disponibilité du résultat est vérifiée sur le canal ADC utilisé (`ADCHS_CH4`). Lorsque la donnée est prête, la valeur brute est récupérée puis envoyée dans la file d'attente FreeRTOS à l'aide de la fonction `xQueueSend`.

L'envoi de la donnée dans la file d'attente permet non seulement de transmettre la valeur à la tâche suivante, mais également de synchroniser naturellement les tâches du système. Une LED est également basculée à chaque acquisition afin de visualiser l'activité de la tâche.

Ce découpage fonctionnel garantit que la tâche d'acquisition reste rapide, déterministe et indépendante des autres traitements. Cette approche est particulièrement adaptée à une application temps réel, car elle assure une acquisition fiable tout en évitant que des traitements plus longs n'affectent la lecture du capteur.

3.2.2 Tâche de conversion de la température

La tâche de conversion, également appelée tâche d'analyse, est responsable du traitement de la donnée brute acquise par la tâche d'acquisition. Son rôle est de transformer la valeur issue du convertisseur analogique-numérique (ADC) en une température exprimée en degrés Celsius, compréhensible par l'utilisateur.

Lors de l'initialisation (`ANALYSE_Initialize`), une file d'attente FreeRTOS est créée afin de stocker les valeurs de température converties, de type `double`. Cette file d'attente permet de transmettre le résultat de l'analyse à la tâche d'affichage. La tâche est ensuite placée dans son état initial.

La fonction `ANALYSE_Tasks` repose sur une machine à états simple comportant deux états

Dans l'état de service, la tâche commence par vérifier la disponibilité de la file d'attente provenant de la tâche d'acquisition. Elle tente ensuite de récupérer une valeur brute (`uint16_t`) à l'aide de la fonction `xQueueReceive`. Si aucune donnée n'est disponible, la tâche reste inactive, ce qui évite toute attente active inutile.

```
case ANALYSE_STATE_SERVICE_TASKS:
{
    uint16_t Buffer0;
    double Tempo;
    if( myQueue == 0 ){
        break;
    }

    if(xQueueReceive(myQueue,&Buffer0,0) == pdPASS){

        Tempo = (((Buffer0*3.3)/4095) - 0.5)*100;
        xQueueSend(myQueue1,&Tempo,10);
        GPIO_RA3_Toggle();

    }
    break;
}
```

Figure 3.3: Tâche de conversion

Lorsque la donnée est reçue avec succès, la conversion est effectuée à l'aide de la relation suivante :

$$T = \left(\frac{V_{ADC} \times 3.3}{4095} - 0.5\right) \times 100$$

Cette formule permet de convertir la valeur numérique de l'ADC en une tension, puis en une température exprimée en degrés Celsius.

Une fois la conversion terminée, la valeur de température est envoyée dans une seconde file d'attente à destination de la tâche d'affichage, à l'aide de la fonction `xQueueSend`. Une LED est également basculée afin de visualiser l'exécution de la tâche de conversion.

3.2.3 Tâche d'affichage

La tâche d'affichage est la dernière étape de la chaîne de traitement du thermomètre temps réel. Son rôle est de récupérer la température convertie fournie par la tâche d'analyse et de l'afficher sur l'écran LCD de la carte Explorer 16/32. Cette tâche est exclusivement dédiée à la gestion de l'interface utilisateur et des périphériques d'affichage.

Lors de l'initialisation (`DISPLAY_Initialize`), la tâche est placée dans son état initial. Le timer `TMR2` est démarré et l'écran LCD est initialisé à l'aide de la fonction `initializeLCD`. Cette étape permet de s'assurer que le périphérique d'affichage est prêt avant toute tentative d'écriture.

```
case DISPLAY_STATE_SERVICE_TASKS:
{

    double temp;
    if( myQueue1 == 0 ){

        break;
    }
    if(xQueueReceive(myQueue1,&temp,0) == pdPASS){
        GPIO_RA6_Toggle();
        DisplayMenu();
            if(GPIO_RD13_Get() ==0){

                displayTemperature(temp);
                Delay_ms(50);
                break;


        }

    }
    break;
}
```

Figure 3.4: Tâche de display

Dans l'état de service, la tâche tente de récupérer une valeur de température de type `double` depuis la file d'attente `myQueue1`, alimentée par la tâche de conversion. Si aucune donnée n'est disponible, la tâche reste inactive, ce qui permet d'éviter toute attente active inutile.

Lorsque la donnée est reçue avec succès, une LED est basculée afin de visualiser l'exécution de la tâche d'affichage. La fonction `DisplayMenu` est ensuite appelée afin de gérer l'interface utilisateur. L'affichage de la température est conditionné par l'état d'un bouton connecté à l'entrée `RD13`. Lorsque ce bouton est activé, la fonction `displayTemperature`

est utilisée pour afficher la valeur de la température sur l'écran LCD.

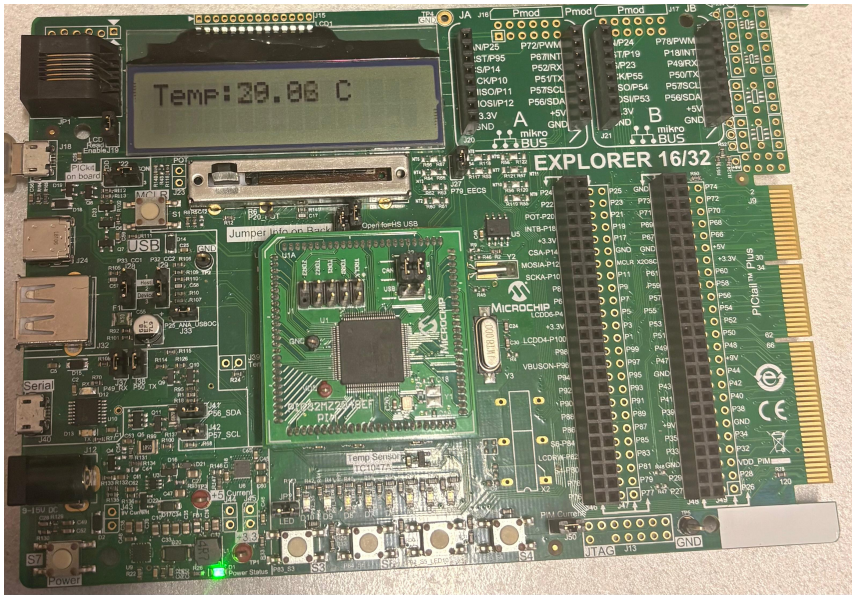


Figure 3.5: Display Temperature

Un délai court est introduit après la mise à jour de l'affichage afin d'améliorer la stabilité visuelle et d'éviter des rafraîchissements trop rapides. Ce choix permet d'assurer un affichage lisible sans perturber le fonctionnement global du système temps réel.

3.2.4 Communication et synchronisation

Dans l'architecture RT-Therm V1, la communication et la synchronisation entre les différentes tâches sont assurées exclusivement à l'aide des mécanismes fournis par FreeRTOS, en particulier les **files d'attente** (*Queues*). Ce choix permet de garantir une architecture modulaire, robuste et adaptée aux contraintes des systèmes temps réel.

Deux files d'attente sont utilisées dans le système :

- une première file d'attente reliant la tâche d'acquisition à la tâche de conversion, utilisée pour transmettre la valeur brute issue de l'ADC ;
- une seconde file d'attente reliant la tâche de conversion à la tâche d'affichage, utilisée pour transmettre la température convertie.

Les files d'attente jouent un double rôle. Elles permettent tout d'abord le **transfert sécurisé des données** entre les tâches, sans partage direct de variables globales. Chaque tâche ne peut accéder qu'aux données qui lui sont explicitement transmises, ce qui réduit les risques d'erreurs et améliore la lisibilité du code.

Grâce à cette approche, la tâche de conversion ne s'exécute que lorsqu'une nouvelle mesure est disponible, et la tâche d'affichage ne s'active que lorsqu'une nouvelle température a été calculée. La synchronisation est ainsi naturellement assurée par la circulation des données, sans nécessiter de mécanismes supplémentaires tels que des sémaphores ou des interruptions.

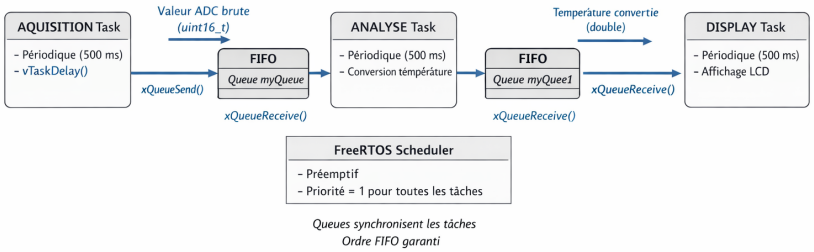


Figure 3.6: Schema fonctionnel des tâches

Cette stratégie de communication événementielle permet d'obtenir un comportement déterministe et prévisible du système. Elle constitue une approche particulièrement adaptée à un premier projet utilisant FreeRTOS, car elle met en évidence les principes fondamentaux de la communication inter-tâches et de la synchronisation dans un système temps réel.

Chapter 4

Architecture de RT-Therm V2

La version RT-Therm V2 constitue une évolution significative par rapport à RT-Therm V1. La contrainte principale imposée dans cette version est l'utilisation d'un **seul délai explicite**, appliqué exclusivement à la tâche d'acquisition de la température. Cette contrainte nécessite une nouvelle stratégie de coordination et de synchronisation entre les tâches.

La solution retenue repose sur une architecture **événementielle**, dans laquelle la synchronisation entre les tâches est assurée principalement par l'utilisation d'un **sémaphore binaire FreeRTOS**. Ce mécanisme constitue l'élément central de RT-Therm V2.

4.1 Problématique et motivation

Dans RT-Therm V1, chaque tâche disposait de son propre délai, ce qui permettait de contrôler simplement leur fréquence d'exécution. Toutefois, cette approche présente certaines limites :

- exécution périodique de tâches même en l'absence de nouvelles données ;
- synchronisation implicite reposant uniquement sur le temps ;
- difficulté à garantir un enchaînement strict entre acquisition, conversion et affichage.

La contrainte de RT-Therm V2 impose donc de repenser complètement la coordination des tâches. L'objectif est de garantir que les tâches de conversion et d'affichage ne s'exécutent **que lorsqu'une nouvelle mesure valide est disponible**.

4.2 Principe général de RT-Therm V2

Dans cette version, la tâche d'acquisition devient la **référence temporelle unique du système**. Elle est exécutée périodiquement à l'aide de la fonction `vTaskDelay`, ce qui fixe précisément la période d'échantillonnage de la température.

Les autres tâches (analyse, affichage et interface utilisateur) ne sont plus pilotées par le temps. Leur exécution dépend exclusivement de la disponibilité des données et des signaux de synchronisation fournis par la tâche d'acquisition.

Cette organisation permet de passer d'une architecture basée sur le temps à une architecture basée sur les événements.

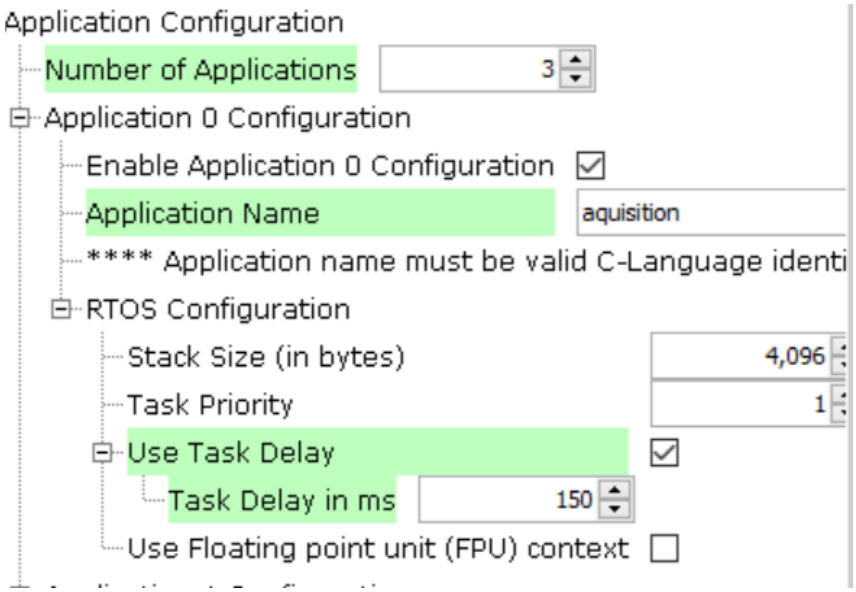


Figure 4.1: Acquisition Config

4.2.1 Rôle central du sémaphore binaire

Dans l'architecture RT-Therm V2, le **sémaphore binaire** constitue le mécanisme principal de coordination et de synchronisation entre les tâches. Il est utilisé pour signaler de manière explicite qu'une nouvelle mesure de température a été acquise et validée, autorisant ainsi la poursuite du traitement.

Contrairement à RT-Therm V1, où la synchronisation reposait principalement sur des délais, RT-Therm V2 adopte une approche événementielle. Le sémaphore permet de matérialiser cet événement : *« une nouvelle donnée est disponible »*.

Le fonctionnement global peut être décrit comme suit :

1. la tâche d'acquisition déclenche une conversion ADC ;
2. lorsque la conversion est terminée, la valeur brute est envoyée dans une file d'attente ;
3. la tâche d'acquisition libère ensuite le sémaphore binaire ;
4. les tâches dépendantes sont alors autorisées à poursuivre leur exécution.

La Figure 4.2 illustre le rôle de la tâche d'acquisition dans ce mécanisme. Après avoir récupéré la valeur issue du convertisseur ADC et l'avoir envoyée dans la file d'attente, la fonction `xSemaphoreGive` est appelée. Cette instruction joue un rôle fondamental : elle signale explicitement au reste du système qu'une nouvelle mesure valide est disponible. Le sémaphore passe alors à l'état « libéré ».

```
case AQUISITION_STATE_SERVICE_TASKS:
{


    ADCHS_GlobalEdgeConversionStart();

    if(ADCHS_ChannelResultIsReady(ADCHS_CH4)){

        uint16_t Buffer = ADCHS_ChannelResultGet(ADCHS_CH4);

        if( myQueue != 0 ){

            if( xQueueSend(myQueue,&Buffer,10) == pdPASS){

                xSemaphoreGive(xmysemaPhore);
                GPIO_RA2_Toggle();
            }
```

Figure 4.2: Libération du sémaphore par la tâche d'acquisition après une conversion ADC valide

La Figure 4.3 montre le comportement de la tâche d'analyse face à ce sémaphore. Cette tâche reste bloquée tant que le sémaphore n'est pas libéré. L'appel à `xSemaphoreTake` permet de suspendre l'exécution de la tâche jusqu'à la réception du signal provenant de la tâche d'acquisition. Une fois le sémaphore obtenu, la tâche d'analyse peut récupérer la donnée depuis la file d'attente et effectuer la conversion de la température.

```
case DISPLAY_STATE_SERVICE_TASKS:
{

    if(xSemaphoreTake(xSemaphoreTemp,0) == pdPASS ){

        if(xQueueReceive(myQueueTemp, &tempMsg, 0) == pdPASS){

            if (GPIO_RD13_Get() == 0)     // Active-low button S6(le 2eme)
            {
                tempMsg.unit = ToggleTempUnit(tempMsg.unit);
                GPIO_RA6_Toggle();

            }

            displayTemperature(tempMsg);


    }
```

Figure 4.3: Attente et prise du sémaphore par la tâche d'analyse avant le traitement

Le sémaphore agit ainsi comme un **signal de validation de la mesure**. Il garantit que les tâches de traitement (analyse et affichage) ne peuvent pas s'exécuter tant qu'une donnée correcte, récente et cohérente n'a pas été produite par la tâche d'acquisition. Cette synchronisation explicite permet d'éviter toute exécution prématurée ou redondante des tâches et assure un enchaînement strict et déterministe des différentes étapes du traitement.

4.2.2 Coordination entre les tâches

Dans RT-Therm V2, la coordination repose sur la combinaison de deux mécanismes complémentaires :

- les **files d'attente**, utilisées pour le transfert des données ;
- le **sémaphore binaire**, utilisé pour la synchronisation et l'ordonnancement logique des tâches.

Cette combinaison permet de séparer clairement les rôles :

- la file d'attente transporte l'information ;
- le sémaphore contrôle le moment où cette information peut être exploitée.

Ainsi, les tâches de conversion et d'affichage restent bloquées tant que le sémaphore n'est pas libéré, ce qui empêche toute exécution prématurée ou redondante.

4.2.3 Gestion du temps et horodatage des mesures

Afin d'enrichir l'information affichée à l'utilisateur, chaque mesure de température est associée à un horodatage. Celui-ci représente le temps écoulé depuis le démarrage du système et est exprimé en secondes.

L'horodatage est obtenu à partir du compteur de ticks du noyau FreeRTOS, via la fonction `xTaskGetTickCount`. Ce compteur est ensuite converti en secondes en tenant compte de la fréquence du tick du système (`configTICK_RATE_HZ`).

Ce choix présente plusieurs avantages :

- aucune dépendance à un module RTC externe ;
- simplicité d'implémentation ;
- cohérence avec l'architecture temps réel basée sur FreeRTOS.

L'horodatage est ensuite formaté par la tâche d'affichage sous la forme `mm:ss` afin de respecter les contraintes de l'écran LCD (16 caractères par ligne) et d'assurer une lecture claire pour l'utilisateur.

Ce mécanisme permet de visualiser l'évolution temporelle des mesures et constitue une première approche de la gestion du temps dans un système embarqué temps réel.

4.2.4 Structure des données échangées

Dans l'architecture RT-Therm V2, les informations échangées entre les tâches ne se limitent plus à une simple valeur de température. Afin de répondre aux besoins fonctionnels de l'application, une structure de données dédiée a été définie pour encapsuler l'ensemble des informations nécessaires à l'affichage et à l'interaction utilisateur.

La structure `TempMessage_t` regroupe :

- la température exprimée en degrés Celsius ;

- la température exprimée en degrés Fahrenheit ;
- l'unité actuellement sélectionnée pour l'affichage ;
- un horodatage représentant le temps écoulé depuis le démarrage du système.

```c
#ifndef _TEMP_MESSSAGE_H
#define _TEMP_MESSSAGE_H

// *****************************************************************************
// *****************************************************************************
// Section: Included Files
// *****************************************************************************
// *****************************************************************************


typedef enum
{
    TEMP_UNIT_CELSIUS = 0,
    TEMP_UNIT_FAHRENHEIT
} TempUnit_t;

/* Temperature message structure (received from queue) */
typedef struct
{
    int16_t value_c;          // temperature value
    TempUnit_t  value_f;
    TempUnit_t unit; // unit of temperature// unit of temperature
    uint32_t    timestamp;      // Time in seconds since boot (or RTC)
} TempMessage_t;
```

Figure 4.4: Structure des données échangées

Cette structure est transmise intégralement via une file d'attente FreeRTOS entre la tâche d'analyse et la tâche d'affichage. Ce choix permet de garantir la cohérence des données affichées et d'éviter toute dépendance directe entre les tâches.

L'utilisation d'une structure unique présente plusieurs avantages :

- regroupement logique des informations liées à une même mesure ;
- réduction du nombre de files d'attente nécessaires ;
- amélioration de la lisibilité et de l'évolutivité du code.

Cette approche est couramment utilisée dans les systèmes temps réel embarqués afin de garantir un échange de données structuré et robuste entre les différentes tâches.

4.2.5 Interaction utilisateur et changement d'unité

La tâche d'affichage intègre également une interaction utilisateur simple via un bouton poussoir connecté à l'entrée `RD13`. Ce bouton permet à l'utilisateur de basculer dynamiquement l'unité d'affichage de la température entre les degrés Celsius et les degrés Fahrenheit.

Lors de l'appui sur le bouton, l'unité d'affichage est modifiée à l'aide d'une fonction dédiée. Cette fonction inverse l'unité courante sans recalculer la température, les deux valeurs étant déjà disponibles dans la structure de données transmise par la tâche d'analyse.

Ce choix de conception permet :

- de séparer clairement le calcul des données de leur présentation ;
- d'éviter des conversions répétées inutiles ;
- d'améliorer la réactivité de l'interface utilisateur.

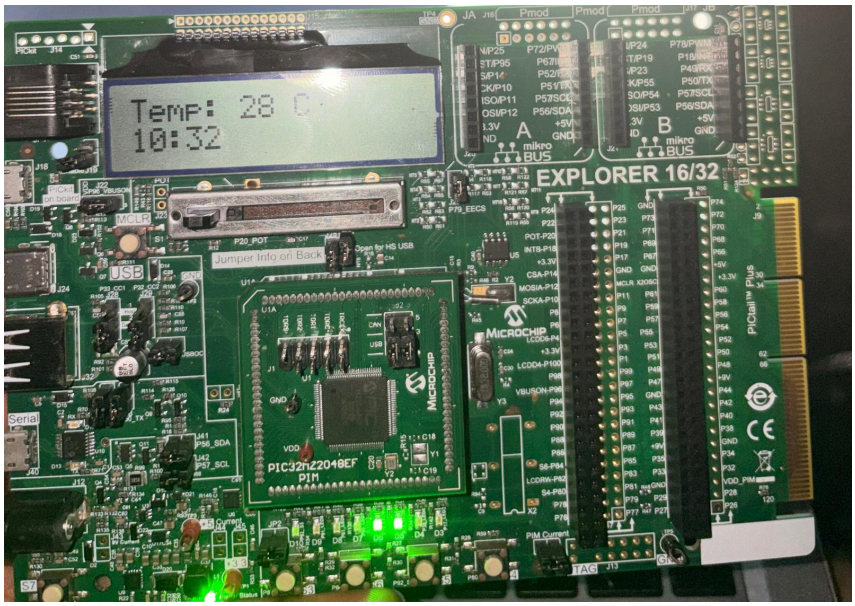


Figure 4.5: Structure des données échangées

Cette interaction illustre l'intégration d'une interface homme-machine simple dans un système temps réel, tout en respectant les principes de modularité et de séparation des responsabilités entre les tâches.

4.2.6 Avantages de la solution retenue

L'utilisation du sémaphore comme élément central de synchronisation présente plusieurs avantages :

- respect strict de la contrainte d'un seul délai explicite ;
- suppression des exécutions inutiles des tâches ;
- meilleure maîtrise de l'enchaînement acquisition → conversion → affichage ;
- comportement déterministe et prévisible du système ;
- architecture typique des systèmes temps réel industriels.

RT-Therm V2 illustre ainsi une approche plus avancée de la programmation temps réel, dans laquelle la synchronisation par événements remplace la temporisation systématique. Le sémaphore joue un rôle fondamental dans cette architecture et constitue la clé de la coordination entre les différentes tâches.

4.3 Comparaison entre RT-Therm V1 et RT-Therm V2

Le tableau suivant synthétise les principales différences entre les architectures RT-Therm V1 et RT-Therm V2, tant du point de vue de la gestion du temps que de la communication et de la synchronisation.

Critère	**RT-Therm V1**	**RT-Therm V2**
Gestion du temps	Chaque tâche possède son propre délai explicite	Une seule tâche (acquisition) est temporisée
Architecture	Basée sur des tâches périodiques	Basée sur des événements
Tâche de référence	Aucune tâche centrale	Tâche d'acquisition (référence temporelle)
Communication inter-tâches	Files d'attente (Queues) uniquement	Files d'attente + sémaphore binaire
Synchronisation	Implicite, basée sur les délais	Explicite, basée sur le sémaphore
Déclenchement des tâches	Périodique, même sans nouvelles données	Conditionné par la disponibilité d'une nouvelle mesure
Risque d'exécutions inutiles	Présent	Supprimé
Déterminisme	Correct	Élevé
Complexité de l'architecture	Simple et pédagogique	Plus avancée et plus proche d'un cas industriel
Objectif pédagogique	Introduction aux tâches et aux Queues	Compréhension de la synchronisation événementielle

Table 4.1: Comparaison des architectures RT-Therm V1 et RT-Therm V2

Chapter 5

Conclusion Générale

Ce projet a permis de mettre en œuvre une application complète de thermomètre embarqué reposant sur un système d'exploitation temps réel. À travers les deux versions proposées, RT-Therm V1 et RT-Therm V2, les principes fondamentaux des systèmes temps réel ont pu être progressivement introduits, compris et appliqués.

La version RT-Therm V1 a constitué une première approche pédagogique de FreeRTOS. Elle a permis de découvrir la création et la gestion de tâches concurrentes, l'utilisation des files d'attente pour la communication inter-tâches ainsi que l'organisation modulaire d'une application embarquée. Cette version met en évidence une architecture simple, basée sur des tâches périodiques, adaptée à une première prise en main d'un RTOS.

La version RT-Therm V2 représente une évolution plus avancée de l'architecture. En imposant l'utilisation d'un seul délai explicite, elle a conduit à l'adoption d'une approche événementielle reposant sur la synchronisation par sémaphore binaire. Ce mécanisme joue un rôle central dans la coordination des tâches et garantit un enchaînement déterministe entre l'acquisition, l'analyse et l'affichage des données. Cette architecture est plus efficace et plus proche des pratiques utilisées dans les systèmes temps réel industriels.

L'utilisation conjointe du framework Harmony V3 et de FreeRTOS a permis de se concentrer sur la conception logicielle et les mécanismes temps réel, tout en simplifiant la configuration matérielle du microcontrôleur. Ce projet a ainsi contribué à renforcer la compréhension des notions clés telles que l'ordonnancement, la synchronisation, la communication inter-tâches et la gestion du temps.

En conclusion, ce TP constitue une introduction solide à la programmation temps réel embarquée. Les compétences acquises au cours de ce travail pourront être réinvesties dans des projets plus complexes intégrant des contraintes temps réel plus strictes, des périphériques multiples ou des architectures logicielles plus élaborées.